

Dynamic Arrays & Analysis

CSCI 235 — Class #4

Tong Yi

Hunter College, CUNY

Thursday, September 10, 2026

From Your Lecture Slips

Answers to the questions you turned in on Tuesday (9/8),
grouped by topic.

Pop quizzes

Several of you asked whether there will be pop quizzes, and whether/how they'd affect grading — this isn't on the syllabus yet. *[to be announced in class]*

Also noted

Requests for more **practical, hands-on coding practice**, more **syntax** (not just concepts), and a **simulation / visual walkthrough** of how the MiniVector examples run step-by-step. All heard — I'll work more of this into class.

The explicit Keyword

Your question

“What’s the explicit keyword?”

explicit on a constructor blocks the compiler from using it for *implicit* conversions:

```
explicit MiniVector(std::size_t count);  
  
MiniVector m(5); // OK -- direct-initialization  
MiniVector m = 5; // ERROR: no implicit int -> MiniVector
```

Why it matters

Without explicit, `MiniVector m = 5;` would silently build a 5-element vector from a bare `int` — surprising and easy to trigger by accident.

Member Initializer List — A Second Look

Your question

“The member initializer list is just attributes of the private variables?” / “I am extremely confused with the syntax.”

Yes — that intuition is right. Each entry pairs one **data member** with the **value** it should start with:

```
MiniVector::MiniVector(std::size_t count)
: data_(new int[count > 0 ? count : 2]{}), // data_ = ...
  size_(count),                          // size_ = ...
  capacity_(count > 0 ? count : 2) {      // capacity_ = ...
}
```

Read it as

“Initialize `data_` with *this*, `size_` with *this*, and `capacity_` with *this*. The members are actually initialized in their **declaration order**, then the constructor body runs.”

Destructors

Your question

“What is a destructor?”

```
class MiniVector {  
    public:  
        ~MiniVector(); // the destructor  
};  
  
MiniVector::~~MiniVector() {  
    delete[] data_; // free what the constructor allocated  
}
```

What it's for

A destructor runs **automatically** when an object's lifetime ends (its scope exits, or it's delete'd). It's where MiniVector gives back the heap array it owns — exactly one destructor per class, no parameters, no return type.

Your question

“What is [the] difference between new / new[]?”

```
int* p = new int(42); // ONE int -> delete p;  
int* arr = new int[10]; // an ARRAY -> delete[] arr;
```

Rule of thumb

new allocates a single object and pairs with delete. new[] allocates an array and pairs with delete[]. Mixing them — delete on an array, or delete[] on a single object — is undefined behavior. MiniVector's data_ always comes from new[], so it always needs delete[].

reserve(): Who Frees the Old Array?

Your question

“How do you forget to free the old array in `reserve()` if it automatically does it?”

It doesn't happen automatically — you write all three steps yourself:

- 1 **Allocate** a new, bigger array with `new[]`.
- 2 **Copy** the existing elements into it.
- 3 **Free** the *old* array with `delete[]`, then point `data_` at the new one.

So it's easy to forget

Skip step 3 and the old array becomes unreachable but never freed — a **memory leak**, not a crash, so nothing tells you it happened.

size_, capacity_, and Growth — Recap

Your questions

“Does capacity just keep doubling?” / “Does the order matter?” / “Size must always be smaller or equal to [capacity]?”

- capacity_ only doubles when push_back needs room it doesn't currently have — not on every push, and not on a timer.
- The invariant is always $\text{size_} \leq \text{capacity_}$: size_ counts *used* slots, capacity_ counts *allocated* slots.
- Yes, order matters — but it's **declaration** order (from the .hpp), not the order you happen to write the initializer list in.

Function Templates: What Is T For?

Your question

“Can you go over the template T again and its purpose?”

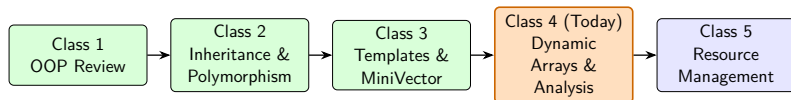
```
template <typename T>
T myMax(T a, T b) {
    return (a > b) ? a : b;
}

myMax(3, 7);           // T = int
myMax(3.5, 2.1);       // T = double
myMax('a', 'z');       // T = char
```

The purpose

T is a **placeholder for a type**, filled in by the compiler at each call site. One template definition replaces writing (and maintaining) a separate `myMax` for every type you need it for.

Where We Are



Today's outline:

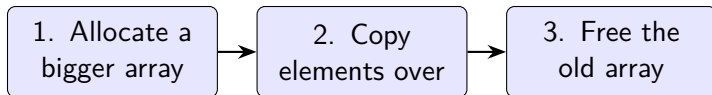
- 1 Recap: why MiniVector resizes, and why doubling wins
- 2 Big-O notation: a precise language for “how fast”
- 3 Three growth rates: $O(1)$, $O(n)$, $O(\log n)$

Recap: The Resize Problem

A raw heap array has a **fixed** size the moment it's allocated:

```
int* data_ = new int[capacity_]; // this size is now fixed
```

There is no way to make it bigger in place. To grow, MiniVector's `reserve()` does the same three steps every time:



(Full details in Class 3 — `reserve()`, `push_back()`.)

Recap: Doubling Gives Amortized $O(1)$ push_back

- push_back calls `reserve(2 * capacity())` only when the array is *already full* — capacity **doubles**, it doesn't just grow by one.
- First 8 pushes from an empty MiniVector: capacity goes 2, 2, 4, 4, 8, 8, 8, 8 — reallocation gets *rarer* exactly as it gets more expensive.
- Total copying work across n pushes is less than a **constant times n** (in fact, under $2n$ with this doubling strategy), so the *average* cost per push is constant, even though a handful of individual calls cost much more.

Today

We make “ $O(1)$ ” and “ $O(n)$ ” precise — and meet a new one, $O(\log n)$.

Why We Need a Language for “How Fast”

- Two algorithms solving the same problem can be compared without running either one — by counting how the number of basic operations **grows** as the input size n grows.
- Big-O describes that growth while ignoring constant factors and lower-order terms — they stop mattering once n is large.

n	$3n + 20$ ops	n^2 ops
5	35	25
50	170	2500
500	1520	250000

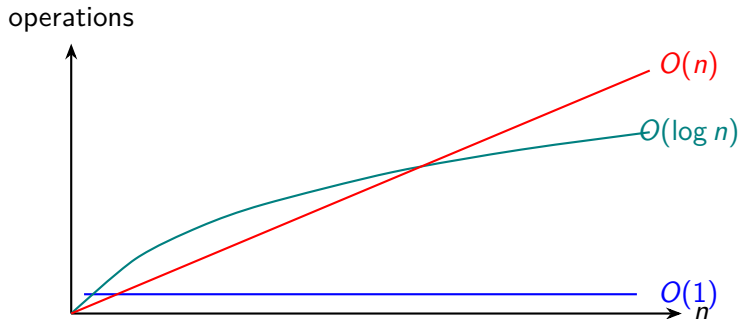
For small n the constants can go either way — but past a certain point, n^2 *always* overtakes any multiple of n . Big-O describes that eventual, large- n behavior: $O(n)$ vs. $O(n^2)$.

Three Growth Rates You'll See Everywhere

Class	Example	Everyday analogy
$O(1)$	<code>data_[i]</code> via <code>operator[]</code>	Looking up a word when you already know the page number
$O(n)$	Scanning an unsorted array for a value, one element at a time	Reading every page of a book to find one fact
$O(\log n)$	Repeatedly halving a <i>sorted</i> search range	"Guess my number, 1–1000," always guessing the midpoint

$O(\log n)$ is a preview — binary search itself is coming up later in the course.

Growth Rates, Visually



- Near the origin, all three look similar — the differences only show up once n gets large.
- That gap, once it opens up, is the entire reason algorithm choice matters for big inputs.

Amortized $O(1)$: A Precise Kind of Average

Now we have the vocabulary to say exactly what “amortized” means:

- **Worst case, single call:** a `push_back` that triggers a reallocation is $O(n)$ — it copies every existing element.
- **Amortized, over a long sequence of calls:** $O(1)$ — spread the occasional $O(n)$ reallocation cost over all the calls since the last one, and each call pays only a constant share.

Not the same as “average case”

Average case is about typical/random *input*. Amortized is a guarantee about a *sequence of operations* — it holds no matter what the inputs are.

Quick Check

Q1. `operator[]` just returns `data_[i]`. Growth rate?

Answer

$O(1)$ — one array access, regardless of `size_`.

Q2. You search an unsorted `MiniVector` by checking every element until you find a match (or reach the end). Growth rate?

Answer

$O(n)$ — in the worst case (no match, or a match at the very end) you check all n elements.

Q3. Is a single reallocating `push_back` call $O(1)$ or $O(n)$?
What about `push_back`'s cost *amortized* across many calls?

Answer

That one call is $O(n)$; amortized across many calls, it's $O(1)$.

What's Due

Item	Posted	Due
Exercise 2 (Pointer Practice)	Tu 9/8	today, Th 9/10
Project 1A: MiniVector	Tu 9/1	Th 9/17

Full details and due dates are always on the course schedule page.

Note

No extra credit is offered this semester.

Key Takeaways

- A raw heap array has a **fixed** size — growing means allocate, copy, free.
- Doubling capacity makes reallocation rare enough that `push_back` is **amortized** $O(1)$.
- **Big-O** describes how work grows with input size n , ignoring constants and lower-order terms.
- Three growth rates to recognize on sight: $O(1)$ (constant), $O(n)$ (linear), $O(\log n)$ (logarithmic).
- **Amortized** is a guarantee about a sequence of calls, not about typical input — don't confuse it with “average case.”

Questions?

See you Tuesday, 9/15 — Resource Management (Rule of Three).